

JAVA PROGRAMMING · DAY 3

Switch, Formulas & Loops

Switch Case with Strings, Area Formulas, for Loop, while Loop

7 Programs · Explained Step-by-Step for Easy Understanding

1 Control Statements (Switch Case with Strings)

On Day 2 we used **switch** with numbers. Java also lets you switch on a **String** value — very handy when the user types words instead of numbers.

Program 1 — Print Day Position Using a String Switch

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the day ");
        String day = sc.next();

        switch (day) {
            case "Monday":
                System.out.println("Day 1 of the week");
                break;
            case "Tuesday":
                System.out.println("Day 2 of the week");
                break;
            case "Wednesday":
                System.out.println("Day 3 of the week");
                break;
            default:
                System.out.println("Invalid day");
        }
    }
}
```

CONSOLE OUTPUT (SAMPLE RUN)

```
Enter the day Wednesday
Day 3 of the week
```

What's happening here?

- **sc.next()** reads a single word (stops at the first space) — perfect for a one-word day name.
- Each **case "Monday":** is compared against the text stored in **day**. The match is **exact and case-sensitive** — typing "monday" (lowercase) would fall through to **default**.
- Only 3 days are handled here (Monday-Wednesday). As practice, try adding **case** blocks for Thursday to Sunday yourself!

2 Find the Area of a Triangle

A common formula-based program: take measurements from the user, apply a formula, and print the result.

Formula: **Area** = $\frac{1}{2} \times \text{base} \times \text{height}$

Program 1 — Area of Triangle

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the base of the triangle: ");
        double base = sc.nextDouble();

        System.out.print("Enter the height of the triangle: ");
        double height = sc.nextDouble();

        double area = 0.5 * base * height;
        System.out.println("Area of Triangle : " + area);
    }
}
```

CONSOLE OUTPUT (SAMPLE RUN)

```
Enter the base of the triangle: 10
Enter the height of the triangle: 6
Area of Triangle : 30.0
```

What's happening here?

- Both **base** and **height** are stored as **double** so the program also works with decimal measurements (like 6.5 cm).
- **sc.nextDouble()** reads a decimal number typed by the user, just like **nextInt()** reads a whole number.
- 0.5 is the same as $\frac{1}{2}$ — multiplying by 0.5 is a common way to write "divide by 2" in code.

3 Find the Area of a Trapezoid

A trapezoid has two parallel sides (call them **a** and **b**) and a height between them.

Formula: **Area** = $\frac{1}{2} \times (a + b) \times \text{height}$

Program 1 — Area of Trapezoid

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the length of side a: ");
        double a = sc.nextDouble();

        System.out.print("Enter the length of side b: ");
        double b = sc.nextDouble();

        System.out.print("Enter the height: ");
        double height = sc.nextDouble();

        double area = 0.5 * (a + b) * height;
        System.out.println("Area of Trapezoid : " + area);
    }
}
```

CONSOLE OUTPUT (SAMPLE RUN)

```
Enter the length of side a: 8
Enter the length of side b: 12
Enter the height: 5
Area of Trapezoid : 50.0
```

What's happening here?

- The parentheses (**a + b**) are important — they make sure the addition happens **before** the multiplication, matching the formula.
- Java follows normal maths order of operations (BODMAS/PEMDAS), so removing the brackets would give a wrong answer.

4 Loops — for Loop

A **loop** repeats a block of code automatically instead of writing the same line again and again. A **for** loop is used when you know exactly how many times to repeat.

Syntax: **for (initialization; condition; update) { ... }**

Program 1 — Loop Execution Message

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Loop Execution : " + i);
        }
    }
}
```

CONSOLE OUTPUT

```
Loop Execution : 1
Loop Execution : 2
Loop Execution : 3
Loop Execution : 4
Loop Execution : 5
```

What's happening here?

- **int i = 1** → starting point (runs once, at the very beginning).
- **i <= 5** → condition checked **before every repeat**; the loop stops as soon as this becomes false.
- **i++** → increases **i** by 1 after each repeat.

Program 2 — Print 1 2 3 4 5 (same line) Using a Loop

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.print(i + " ");
        }
    }
}
```

CONSOLE OUTPUT

```
1 2 3 4 5
```

What's happening here?

- **System.out.print** (no "ln") does **not** move to a new line, so every number appears on the same line, one after another.
- Adding " " after **i** puts a space between the numbers so they don't run together as "12345".

Program 3 — Print 1 to 5 (each on a new line) Using a Loop

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}
```

CONSOLE OUTPUT

```
1
2
3
4
5
```

What's happening here?

- This time we use **println**, which **does** move to a new line after printing — so each number lands on its own row.
- Compare this with Program 2: same loop, just **print** vs **println** changes the whole layout of the output.

5 Loops — while Loop

A **while** loop repeats a block of code **as long as** a condition stays true. Unlike **for**, the initialization and update are written separately, giving you more flexibility — useful when you don't know the exact number of repeats in advance.

Syntax: **initialization;** then **while (condition) { ...code...; update; }**

🔍 **Try it yourself:** search "while loop syntax java" on Google or ask ChatGPT, and compare what you find with the structure above.

Program 1 — Print Numbers from 1004 to 1020 in Steps of 4

Expected Output: 1004 1008 1012 1016 1020

```
public class Main {  
    public static void main(String[] args) {  
        int i = 1004;  
  
        while (i <= 1020) {  
            System.out.print(i + " ");  
            i = i + 4;  
        }  
    }  
}
```

CONSOLE OUTPUT

```
1004 1008 1012 1016 1020
```

What's happening here?

- **int i = 1004;** → the starting value is set **before** the loop begins (initialization).
- **while (i <= 1020)** → the loop keeps running as long as **i** is 1020 or less.
- **i = i + 4;** → the update step, written manually inside the loop body (this is what makes **while** different from **for**).
- Trace it: 1004 → 1008 → 1012 → 1016 → 1020 → then **i** becomes 1024, the condition **1024 <= 1020** is false, so the loop stops.